

Algorithm Documentation: Parallel Base Optimizer

Average Manipulability Index and Grid Search Strategy
for TM5-700 Surgical Robot Base Optimization

RMML Lab — Isaac Lab Reachability Map Project

May 14, 2026

1 Overview

The `parallel_base_optimizer.py` script performs a **2D grid search** over candidate robot base positions using Isaac Lab’s parallel environment framework. For each candidate position, it evaluates the robot’s ability to reach a set of surgical waypoints and computes a kinematic quality metric — the **Average Manipulability Index (AMI)** — to determine the optimal base placement for a TM5-700 robotic arm in a surgical scenario.

2 Coordinate System and Base Positioning

2.1 USD Hierarchy

The robot exists within a nested USD (Universal Scene Description) hierarchy:

```
/Root
|-- Zero                (global origin reference)
+-- robotarm_base       (grid search target)
    +-- robotarm_base   (second-level Xform)
        +-- tm5_700     (articulation root)
```

Isaac Lab controls the **articulation root** (`tm5_700`) via physics APIs, while the user specifies the desired position of `robotarm_base` (first level). A fixed offset $\mathbf{T}$ relates the two:

$$\mathbf{p}_{\text{tm5}} = \mathbf{p}_{\text{base}} + \mathbf{T} \quad (1)$$

where $\mathbf{T} \in \mathbb{R}^3$ is computed at runtime:

$$\mathbf{T} = \mathbf{p}_{\text{tm5}}^{\text{default}} - \mathbf{p}_{\text{base}}^{\text{default}} \quad (2)$$

For the current USD scene, $\mathbf{p}_{\text{base}}^{\text{default}} = (0.2, 0, 0)^T$ m.

2.2 Grid Search Definition

The search space is defined as a 2D Cartesian grid:

$$\mathcal{G} = \{(x_i, y_j) \mid x_i \in \text{linspace}(x_{\min}, x_{\max}, N_x), y_j \in \text{linspace}(y_{\min}, y_{\max}, N_y)\} \quad (3)$$

Each grid point (x_i, y_j) represents a candidate `robotarm_base` position. The total number of parallel environments is $|\mathcal{G}| = N_x \times N_y$.

3 Jacobian Matrix and Manipulability Index

3.1 Jacobian Matrix

For a 6-DOF serial manipulator with joint configuration $\mathbf{q} \in \mathbb{R}^6$, the **geometric Jacobian** $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 6}$ relates joint velocities to end-effector (needle tip) velocities:

$$\dot{\mathbf{x}} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}} \quad (4)$$

where $\dot{\mathbf{x}} = [\mathbf{v}^T, \boldsymbol{\omega}^T]^T$ contains the linear and angular velocities of the end-effector.

In the script, the Jacobian is extracted directly from the PhysX simulation:

```
J = robot.root_physx_view.get_jacobians()
[:, ee_jacobi_idx, :, joint_col_ids]
```

This retrieves the 6×6 sub-matrix corresponding to the needle tip body and the 6 active joints (joint_1 through joint_6).

3.2 Yoshikawa Manipulability Index

The **Manipulability Index** w , introduced by Yoshikawa (1985)[1], is defined as:

$$w(\mathbf{q}) = |\det(\mathbf{J}(\mathbf{q}))| \quad (5)$$

3.2.1 Geometric Interpretation

The value $w(\mathbf{q})$ represents the **volume of the velocity ellipsoid** — the set of all achievable end-effector velocities for unit joint velocities:

$$\mathcal{E}(\mathbf{q}) = \{\dot{\mathbf{x}} \in \mathbb{R}^6 \mid \|\dot{\mathbf{q}}\| \leq 1, \dot{\mathbf{x}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}\} \quad (6)$$

- $w \rightarrow 0$: The ellipsoid collapses along one or more axes $\Rightarrow$ the robot is near a **kinematic singularity**, losing the ability to move in certain Cartesian directions.
- w large: The ellipsoid is “full” $\Rightarrow$ the robot can move and rotate the end-effector effectively in all directions — high **dexterity**.

3.2.2 Relation to Singular Values

The manipulability index can also be expressed in terms of the singular values σ_i of $\mathbf{J}$:

$$w = \prod_{i=1}^6 \sigma_i \quad (7)$$

If any $\sigma_i = 0$, the robot is at a singularity and $w = 0$.

4 Average Manipulability Index (AMI)

For a given base position $\mathbf{p}_k = (x_k, y_k)$ and a set of M task waypoints $\{\mathbf{w}_1, \dots, \mathbf{w}_M\}$, define:

- $\mathcal{S}_k \subseteq \{1, \dots, M\}$: the set of waypoints **successfully reached** (positioning error $< \epsilon$, where $\epsilon = 1.5$ mm by default).
- $\mathbf{q}_k^{(j)}$: the joint configuration when waypoint $j \in \mathcal{S}_k$ is reached.

The **Average Manipulability Index** for base position k is:

$$\text{AMI}_k = \frac{1}{|\mathcal{S}_k|} \sum_{j \in \mathcal{S}_k} w(\mathbf{q}_k^{(j)}) = \frac{1}{|\mathcal{S}_k|} \sum_{j \in \mathcal{S}_k} \left| \det(\mathbf{J}(\mathbf{q}_k^{(j)})) \right| \quad (8)$$

The script also records the **minimum manipulability**:

$$w_{\min,k} = \min_{j \in \mathcal{S}_k} w(\mathbf{q}_k^{(j)}) \quad (9)$$

This captures the worst-case dexterity across all reachable waypoints.

4.1 Interpretation for Base Selection

The optimal robot base position $\mathbf{p}^* \in \mathcal{G}$ is chosen to maximize a composite fitness score (fg), ensuring both strict reachability across all task waypoints and continuous high dexterity along the trajectory:

$$\mathbf{p}^* = \arg \max_{\mathbf{p}_{i,j}} [fg] \quad (10)$$

Where the fitness score fg for a given base position $\mathbf{p}_{i,j}$ is defined as:

$$fg = S_r^{\min} \cdot \left(\prod_{k=1}^m c(t, k) \right) \cdot cs \quad (11)$$

- $S_r^{\min} = \min_k(RI_k)$: The minimum **Reachability Index (RI)** across all m target waypoints for the base position. $RI_k = \frac{\# \text{ successful orientations}}{\text{total orientations attempted}}$ for waypoint k . This acts as a strict penalty constraint; if any single waypoint is completely unreachable ($RI_k = 0$), the entire base position is deemed invalid ($fg = 0$), ensuring no dead-zones exist in the surgical task space.
- $c(t, k) = \text{AMI}_k$: The dexterity cost function for the k -th waypoint ($1 \leq k \leq m$), defined as the **Average Manipulability Index (AMI)** across all **successful** local orientations:

$$\text{AMI}_k = \frac{1}{|\mathcal{S}_k|} \sum_{\text{successful orientations}} w(\mathbf{q}) \quad (12)$$

where $\mathcal{S}_k$ is the set of successful joint configurations (i.e., those reaching waypoint k within error tolerance ϵ). This represents the average dexterity only among reachable configurations; unreachable configurations contribute zero.

- $cs = \frac{\# \text{ successful tasks}}{\text{total tasks}}$: A scaling coefficient (success rate across all waypoints and orientations) to balance the overall fitness.

4.1.1 Terminology Clarification: RI vs MI vs AMI

- **Manipulability Index (MI, w)**: Defined per joint configuration $\mathbf{q}$ as $w(\mathbf{q}) = |\det(\mathbf{J}(\mathbf{q}))|$. Scalar value ≥ 0 .
- **Reachability Index (RI, RI_k)**: Defined per waypoint k as the fraction of local orientations that successfully reach the waypoint. Range $[0, 1]$.
- **Average Manipulability Index (AMI, AMI_k)**: Defined per waypoint k as the average MI across only the successful joint configurations for that waypoint. Represents dexterity quality among reachable poses.

The product $\prod_{k=1}^m c(t, k) = \prod_{k=1}^m \text{AMI}_k$ aggregates dexterity across the entire trajectory, penalizing waypoints with low average manipulability even if they are technically reachable (high RI_k).

In cases where multiple base positions yield identical fg scores (e.g., highly symmetric configurations or capped metrics), the secondary tie-breaker objective is to maximize the Average Manipulability Index across the entire workspace (Avg_w):

$$\text{If } fg(\mathbf{p}_A) = fg(\mathbf{p}_B), \text{ select } \arg \max(Avg_w) \quad (13)$$

5 Algorithm

Algorithm 1 Parallel Base Position Optimization

Require: Grid parameters $(x_{\min}, x_{\max}, N_x, y_{\min}, y_{\max}, N_y)$, waypoints $\{\mathbf{w}_j\}_{j=1}^M$, threshold ϵ

Ensure: $\text{AMI}_k, |\mathcal{S}_k|$ for each grid point k

```

1:  $\mathcal{G} \leftarrow \text{meshgrid}(x_{\min} : x_{\max}, y_{\min} : y_{\max})$ 
2: Create  $|\mathcal{G}|$  parallel environments in Isaac Lab
3:  $\mathbf{T} \leftarrow \mathbf{p}_{\text{tm5}}^{\text{default}} - \mathbf{p}_{\text{base}}^{\text{default}}$  ▷ Fixed offset
4: for each env  $k = 0, \dots, |\mathcal{G}| - 1$  do
5:   Set  $\mathbf{p}_{\text{tm5}}^{(k)} \leftarrow \mathbf{p}_{\text{env\_origin}}^{(k)} + \mathbf{p}_k + \mathbf{T}$  ▷ Eq. 1
6: end for
7: WRITE_ROOT_POSE_TO_SIM ▷ Single GPU-compatible call
8: Settle joints to initial configuration (50 steps)
9: for each waypoint  $j = 1, \dots, M$  do
10:   Compute target  $\mathbf{w}_j^{(k)}$  for each env (shifted by env origin)
11:   for step = 1, ..., max_steps do
12:      $\mathbf{J}^{(k)} \leftarrow \text{GET\_JACOBIANS}$  for all envs
13:      $\Delta \mathbf{q}^{(k)} \leftarrow \text{DIFFERENTIALIK}(\mathbf{w}_j^{(k)}, \mathbf{J}^{(k)})$ 
14:     Apply joint position targets with clamping
15:     SIM.STEP
16:     if all envs converged ( $\|\mathbf{e}\| < \epsilon$ ) then
17:       break
18:     end if
19:   end for
20:    $d^{(k)} \leftarrow \|\mathbf{p}_{\text{ee}}^{(k)} - \mathbf{w}_j^{(k)}\|$ 
21:   if  $d^{(k)} \leq \epsilon$  then
22:      $|\mathcal{S}_k| += 1$ 
23:      $\text{manip\_sum}_k += |\det(\mathbf{J}^{(k)})|$ 
24:   end if
25: end for
26:  $\text{AMI}_k \leftarrow \text{manip\_sum}_k / |\mathcal{S}_k|$  ▷ Eq. 8
27: Export CSV and generate heatmap
```

6 Differential IK Controller

The script uses the **Damped Least Squares (DLS)** variant of differential inverse kinematics:

$$\Delta \mathbf{q} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \Delta \mathbf{x} \quad (14)$$

where $\lambda = 0.01$ is the damping factor and $\Delta \mathbf{x} = \mathbf{x}_{\text{target}} - \mathbf{x}_{\text{current}}$ is the Cartesian error. This avoids numerical instability near singularities compared to the pseudo-inverse method.

An integral correction term is added when the end-effector is close to the target ($\|\Delta\mathbf{x}\| < 30$ mm):

$$\Delta\mathbf{x}_{\text{boosted}} = \Delta\mathbf{x} + \mathbf{I}_{\text{cart}}, \quad \mathbf{I}_{\text{cart}} = \text{clamp}\left(\sum_t 0.02 \cdot \Delta\mathbf{x}_t, -0.05, 0.05\right) \quad (15)$$

This acts as a proportional-integral (PI) controller in Cartesian space, improving steady-state accuracy.

7 Joint Clamping

After computing $\Delta\mathbf{q}$, joint position targets are clamped to prevent unsafe configurations:

$$q_i^{\text{target}} = \text{clamp}(q_i^{\text{target}}, q_i^{\min}, q_i^{\max}) \quad (16)$$

Table 1: Joint limits applied in the optimizer

Joint	$q_{\min}$ (rad)	$q_{\max}$ (rad)
joint_1	−3.14	−0.1
joint_2	−0.3	2.5
joint_3	−0.3	2.5
joint_4	−0.5	3.14
joint_5	−1.57	1.57
joint_6	−3.14	0.5

These limits are task-specific constraints (not hardware limits) designed to keep the arm in a feasible workspace for the surgical scenario.

References

- [1] T. Yoshikawa, “Manipulability of robotic mechanisms,” *The International Journal of Robotics Research*, vol. 4, no. 2, pp. 3–9, 1985.
- [2] C. P. Sundaram *et al.*, “Robot-Assisted Surgical System Configuration Mapping and Procedure Planning,” referenced in project plan for RASSCMAP cost function and reachability analysis, 2022.